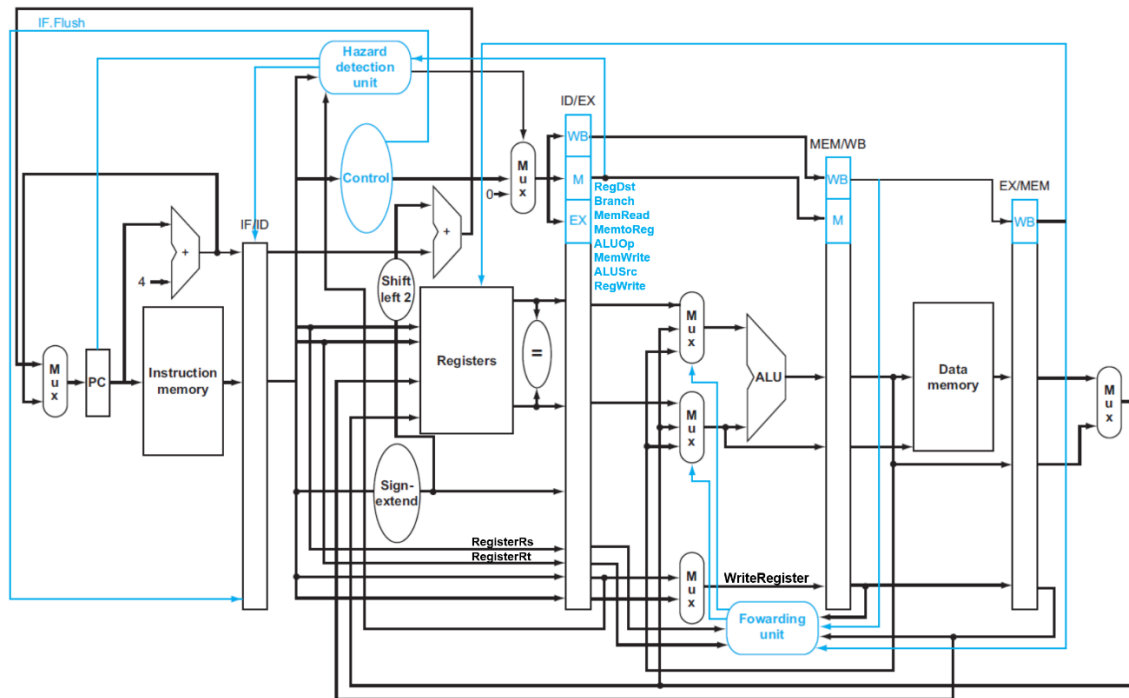


Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
In-Class Exercise (10/15)

- Consider our five-stage pipeline with **full forwarding** and **RF bypassing** shown below (COD Figure 4.65). It now supports branch instructions via a **predict-not-taken** policy, and branches are **resolved at the ID stage**.



You run the following program on this pipelined processor. Note that there are **no data hazards**. The outcomes of branch instructions are provided for you. In total **how many cycles** does it take to run this program to completion; i.e., how many cycles span between (and including) the IF stage of the first instruction and the WB stage of the last? Blank pipeline diagrams have been provided for you on the last page for scratch work if needed; you are not required to fill them in.

```
L1: bne $zero, $t0, L3 <taken>
L2: subi $t1, $t2, 1
L3: bne $zero, $t3, L5 <not taken>
L4: beq $zero, $t4, L7 <taken>
L5: lw $t5, 0($t6)
L6: sw $t7, 0($t8)
L7: ori $t9, $zero, $t9
```

Answer:

10 cycles. Instructions L2 and L5 are fetched but converted to NOPs, while Instruction L6 is not fetched.

inst	clock cycle																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
L1	F	D	X	M	W													
L2		F	=															
L3			F	D	X	M	W											
L4				F	D	X	M	W										
L5					F	=												
L7						F	D	X	M	W								

2. Consider the same pipelined processor as in the previous question (COD Figure 4.65). Note that only RF bypassing is available to the branch decision circuit in the ID stage, **not data forwarding**, so stalls (NOPs) need to be inserted to avoid data hazards with branch source registers. Below is the condition logic used by the Hazard Detection Unit to enable all **data hazard stalls** (including load-to-use stalls) in this pipeline. There are two broad sets of conditions (Condition 1 and Condition 2) highlighted below; a NOP is inserted into the ID/EX pipeline register if either is true.

```

if (
    ( (ID/EX.MemRead || (IF/ID.Branch && ID/EX.RegWrite))
      && (ID/EX.WriteRegister != 0)
      && (ID/EX.WriteRegister == IF/ID.RegisterRs
          || ID/EX.WriteRegister == IF/ID.RegisterRt)
    )
    ||
    ( (IF/ID.Branch && EX/MEM.RegWrite)
      && (EX/MEM.WriteRegister != 0)
      && (EX/MEM.WriteRegister == IF/ID.RegisterRs
          || EX/MEM.WriteRegister == IF/ID.RegisterRt)
    )
) insert NOP at ID/EX; // data hazard; stall instruction in ID

```

Condition 1

OR

Condition 2

You run the following MIPS program on your processor. The **pipeline diagram** is provided for you. State **which cycle(s)** incur a **data hazard stall** and specify whether it is triggered by Condition 1 or Condition 2.

```

L1: addi $t0, $zero, 4
L2: lw   $t1, 0($t0)
L3: add  $s1, $s1, $t1
L4: subi $t0, $t0, 4
L5: bne  $t0, $zero, L2

```

instruction	clock cycle											
	1	2	3	4	5	6	7	8	9	10	11	12
L1	F	D	X	M	W							
L2		F	D	X	M	W						
L3			F	D*	D	X	M	W				
L4				F*	F	D	X	M	W			
L5						F	D*	D*	D	X	M	W

Answer:

Cycle 4: Condition 1 (load-to-use)

Cycle 7: Condition 1 (branch stall; one input will be written by isn in X)

Cycle 8: Condition 2 (branch stall; one input will be written by isn in M)

Remember that the only forwarding path to a branch instruction is W→D RF Bypass.

3. Putting it all together. Consider our pipelined processor from the previous questions (COD Figure 4.65) with full forwarding, RF bypassing, predict-not-taken policy, and branches are resolved at the ID stage. You run the following MIPS program on your processor. Fill in the **pipeline diagram** below for the program's execution. Be sure to consider **both** data hazards and control hazards.

```

L1: addi $t0, $zero, 4
L2: addi $s5, $zero, 20
L3: beq  $t0, $zero, L8
L4: subi $t0, $t0, 4
L5: lw   $t1, 4($t0)
L6: add  $s6, $s6, $t1
L7: bne  $t0, $s5, L3
L8: or   $s7, $s7, $t7

```

inst	clock cycle																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
L1	F	D	X	M	W													
L2		F	D	X	M	W												
L3			F	D*	D	X	M	W										
L4				F*	F	D	X	M	W									
L5						F	D	X	M	W								
L6							F	D*	D	X	M	W						
L7								F*	F	D	X	M	W					
L8										F	=							
L3											F	D	X	M	W			
L4												F	=					
L8													F	D	X	M	W	

L3 has D* stall in cycle 4 for Question2's condition2. Since L3 is D*, L4 must be F* in the same cycle. L6 had D* stall in cycle 8 due to load-to-use stall. L7 is F* in the same cycle. L7 is taken, so L8 is fetched and flushed. L3 is taken, so L4 is fetched and flushed.

Blank pipeline diagrams:

[illegible][illegible]